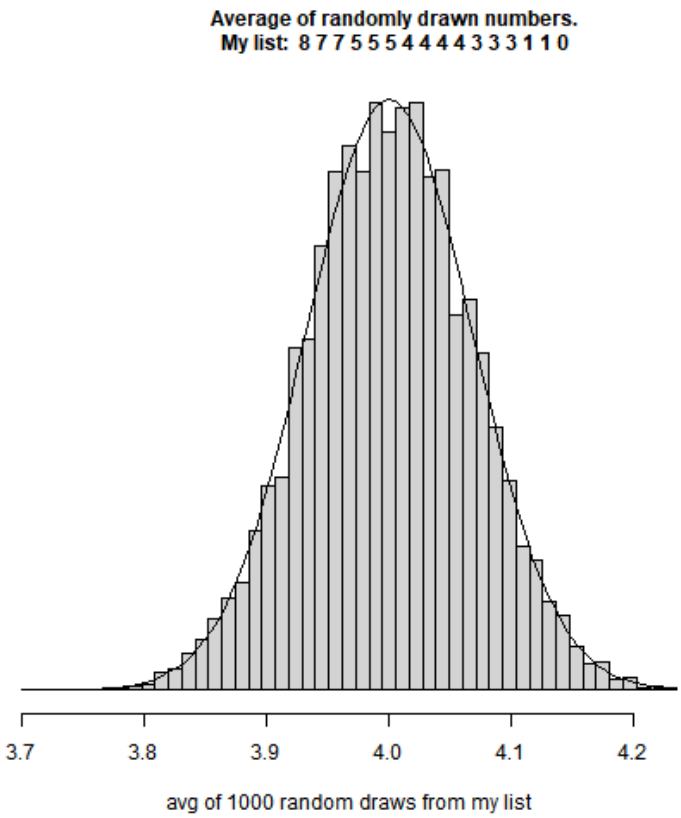
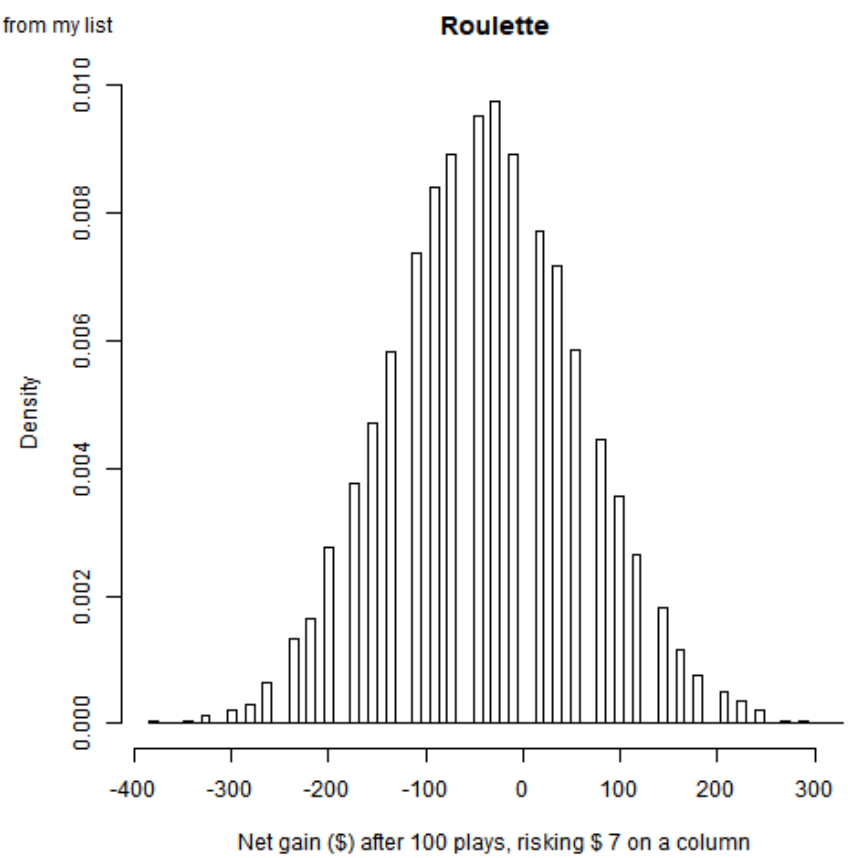
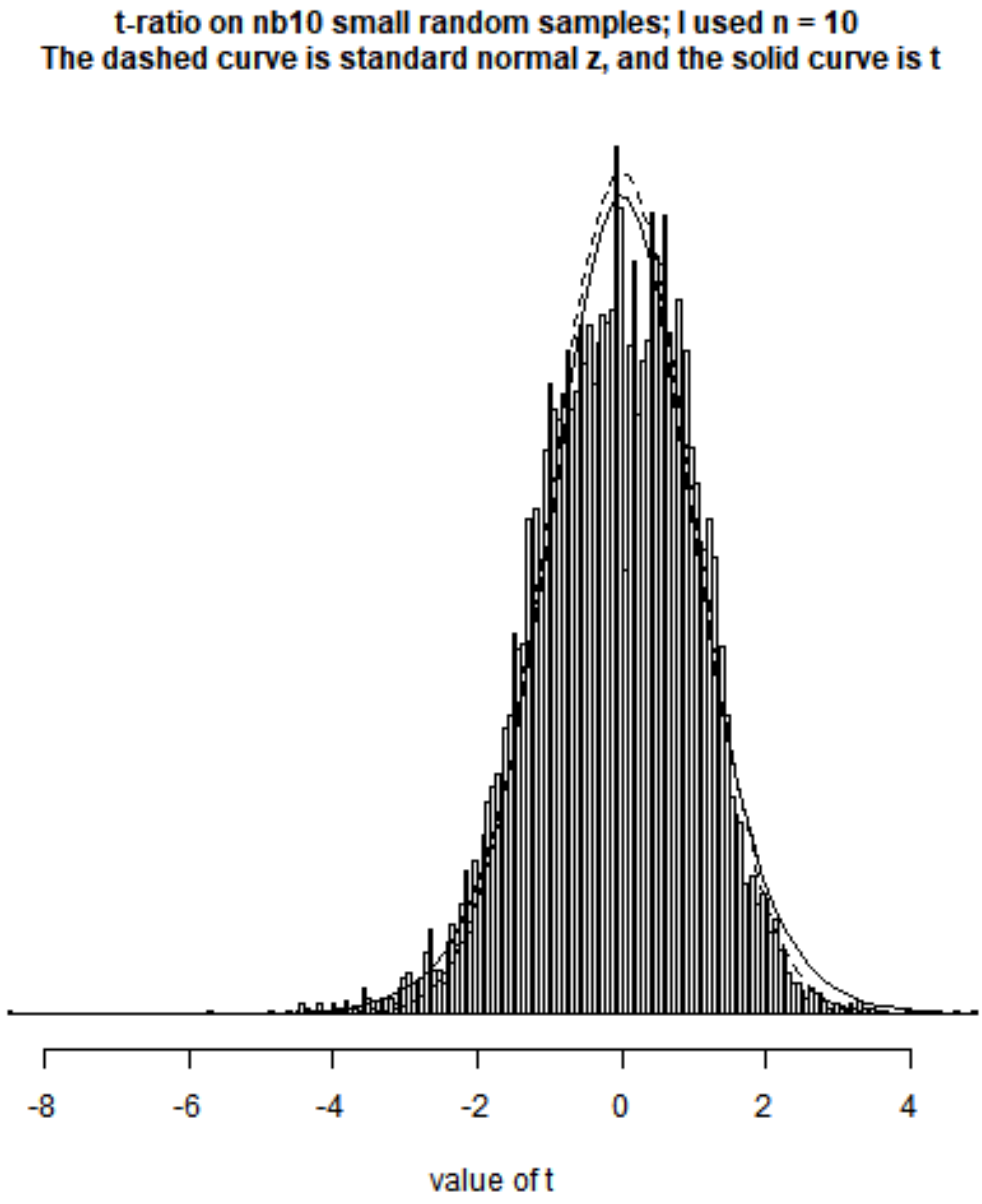
Analysis: Mini-Project 03: Random Variable Simulation

Summary: Use data software to simulate a chance procedure and make a histogram to get an idea of what the true probability histogram might look like.






t-ratio on nb10 small random samples; I used n = 10
The dashed curve is standard normal z, and the solid curve is t
-8
-6
-4
-2
0
2
4
value of t

References

Whalen (PhD), Tristan. (2023). Fall 2023 Lectures 10 and 11 Notes and R Scripts. BUAM 6359.005 - Advanced Statistics for Data Science. University of Texas at Dallas.

```
# Title:   R Script for Mini-Project 03 - Simulation ----
# Author:  KIRPATRICK DORSEY, MSBA Graduate Student ----
# Date:     2023-12-07

# Note:    This work is best viewed in RStudio, as an R script, using the
#          'outline' view.

# Summary: ....----
#          Provided by Dr. Tristan Whalen, PhD, Professor BUAN 6359.005
#
# Use data software to simulate a chance procedure and make a histogram to get
# an idea of what the true probability histogram might look like.
#
# Your submission will feature a total of 3 histograms:
#
#   THE AVERAGE OF A BUNCH OF RANDOMLY DRAWN NUMBERS
#   Pick your own list of numbers. Draw several times at random, with
#   replacement, and take the average of the drawn numbers.
#
#   ROULETTE
#   Pick an amount to risk and a set of slots to risk it on, then play several
#   times, repeating the bet each time. Tally the net gains.
#
#   RATIO
#   An average of randomly drawn numbers, minus its expected value, divided by
#   its standard error, is a random variable used in hypothesis testing. It is
#   the random variable equivalent of standard units or “z-scores” for data
#   values. Depending on the circumstances, it might be called a z-ratio or a
#   t-ratio. Simulate getting a small sample of measurements in a laboratory
#   (using the nb10 data as an example population), and set up the t-ratio for
#   each small sample.

# ************************************************************************** ----
# Tech Stack ----
##           R version 4.3.1 (2023-06-16 ucrt) ----
#           -- "Beagle Scouts"
#           Platform: x86_64-w64-mingw32/x64 (64-bit)
#
##           RStudio 2023.09.0+463 "Desert Sunflower" ----

# ************************************************************************** ----
# Output ----
##       See the ./figures directory ----
#        - avgOfRandNums.png
#        - roulette.simulation.png
#        - t.ratio.simulation.png
# Create 'figures' directory to store analysis output.
dir.create("figures")

# ************************************************************************** ----
# Useful Functions and Utilities ----

## Clear all plots ----
# dev.off(dev.list()["RStudioGD"])

## Clear Global Environment ----
# rm(list = ls())
```

```r
# Dr. Whalen's Functions and Templates ----

# same as "mean" but not as mean
avg <- function(x){sum(x)/length(x)}

# Standard Deviation of a List
# The root-mean-square deviation of a list of numbers.
SD<-function(v){sqrt(sum((v-mean(v))^2)/length(v))}


# ************************************************************************** ----
# Responses ----

# _ ----
## Simulation 1 - AVERAGE OF... NUMBERS ----

### Requirement 1:  Make a list of numbers... ----
# 1) Make a list of numbers. Type it in your document and store it in your
#    R script. Repeats are fine, but please include at least 5 different numbers
#    in your list. Don't use the same list as someone else.
# --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --
#
#### Data: my.list.of.numbers ----
my.list.of.numbers <- c(8,7,7,5,5,5,4,4,4,4,3,3,3,1,1,0)
length(my.list.of.numbers)
# [1] 16
unique(my.list.of.numbers)
# [1] 8 7 5 4 3 1 0
length(unique(my.list.of.numbers))
# [1] 7
#
# Test data from Dr. Whalen's Example
dr.whalens.list <- c(-20,-5,-5,-1,-1,-1,-1,0,1,1,1,1,5,5,20)


### Requirement 2:  Pick the number of times... ----
# 2) Pick the number of times n you will randomly draw (with replacement) from
#    your list. You want it large enough for an approximately normal histogram.
# --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --
#
#### n.draws <- 1000 ----
n.draws <- 1000


### Requirement 3:  Write a simulation in R... ----
# 3) Write a simulation in R that randomly draws n times, with replacement,
#    from your list, then takes the average of the n drawn numbers.
#
# (The average is the constant multiple 1/n of the sum.)
# --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --
#
my.avg.random.draws<-function(numerical.input=c(0,1,2,3,4,5),draws=10,reps=100)
  {
  my.simulated.data <- 1:reps
  for(k in my.simulated.data){
    my.simulated.data[k] <- avg(sample(numerical.input,size=draws,replace=TRUE))
  }
```

```
  # Test output
  my.simulated.data
}
# Example Test Results
test.data <- my.avg.random.draws()
test.data
#
avg(c(0,1,2,3,4,5))
# [1] 2.5
avg(test.data)
# [1] 2.52
min(test.data)
# [1] 1.4
max(test.data)
# [1] 3.7


### Requirement 4:  Repeat the simulation... ----
# 4) Repeat the simulation at least 10,000 or more times.
#
# (More is better, so that the simulation histogram is close to the true
#  probability histogram.)
# --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --
#
# Example Test Results
test.data <-
  my.avg.random.draws(numerical.input=dr.whalens.list,draws=64,reps=10000)
head(test.data)
max(test.data)
# [1] 4.171875
min(test.data)
# [1] -3.890625
avg(test.data)
# [1] 0.01202344


### Requirement 5:  Make a density histogram... ----
# 5) Make a density histogram of the results of the simulations. Use at least 50
# bins so the shape is apparent. If it's not approximately normal, increase the
# number of draws in step 2.
# --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --
#
my.avg.random.draws<-function(
    numerical.input=c(0,1,2,3,4,5),draws=10,reps=100,bins=50)
{
  my.simulated.data <- 1:reps
  for(k in my.simulated.data){
    my.simulated.data[k] <- avg(sample(numerical.input,size=draws,replace=TRUE))
  }
  hist(my.simulated.data,
       prob=TRUE,
       breaks=seq(from=min(my.simulated.data),
                  to=max(my.simulated.data),
                  length.out=bins)
       )
}
#
# Example Test Results
my.avg.random.draws(
```

```
  numerical.input=dr.whalens.list,draws=64,reps=10000,bins=50)


### Requirement 6:  Add the appropriate normal curve... ----
# 6) Add the appropriate normal curve to the histogram. This includes computing,
# within your R script, the expected value and standard error of the random
# variable you’re simulating.
#
# (Remember you have to make a density histogram, or you won’t see the normal
#  curve appear in your plot!)
# --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --
#
# Final histogram, inspired by Lectures 10 and 11 from Dr. Whalen
my.avg.random.draws<-function(
   numerical.input=c(0,1,2,3,4,5),draws=10,reps=100,bins=50)
{
  my.expected.value <- avg(numerical.input)
  my.standard.error <- (sqrt(draws)*SD(numerical.input))/draws
  my.simulated.data <- 1:reps
  for(k in my.simulated.data){
    my.simulated.data[k]<-avg(sample(numerical.input,size=draws,replace=TRUE))
  }
  hist.values <- hist(my.simulated.data,
      prob=TRUE,
      breaks=seq(from=min(my.simulated.data),
              to=max(my.simulated.data),
              length.out=bins),
      main=paste("                Average of randomly drawn numbers.
              My list: ", paste(numerical.input,collapse=" ")),
      cex.main=1,
      xlab= paste("avg of", draws,"random draws from my list"),
      yaxt='n',
      ylab=NULL)
    curve(
    dnorm(x, mean=my.expected.value, sd=my.standard.error),
    add = TRUE)
}
# Example Test Results
my.avg.random.draws(
  numerical.input=dr.whalens.list,draws=64,reps=10000,bins=50)

# Final histogram
my.avg.random.draws(
  numerical.input=my.list.of.numbers,draws=n.draws,reps=10000,bins=50)

# Save the results of Simulation 1
png(filename = "./figures/avgOfRandNums.png", width = 500,height = 500)
  my.avg.random.draws(
    numerical.input=my.list.of.numbers,draws=n.draws,reps=10000,bins=50)
dev.off()

# --- --- --- --- ---
# rm(dr.whalens.list,my.list.of.numbers,n.draws,test.data,my.avg.random.draws)
# --- --- --- --- ---


# _ ----
## Simulation 2: ROULETTE ----
# Use the roulette rules as described in Lesson 10 or in Freedman Ch.16.
```

```
# (In particular, use a 38-slot roulette.)


### Requirement 1:  Pick a whole dollar amount to risk... ----
# 1) Pick a whole dollar amount, other than $1, to risk each play at roulette.
# --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --
#
#### my.money.each.play <- 7 ----
my.money.each.play <- 7


### Requirement 2:  Decide how to bet... ----
# 2) Decide whether to bet on Red, on a Column, or on a Single Slot. If you feel
#    adventurous, you can choose a bet with a different payout: Split, Three
#    numbers, Four numbers, Six numbers.
#
#    (In this case, you find or determine the payout on your own.)
# --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --
#
#### Betting on a Column ----


### Requirement 3:  Choose your number of plays... ----
# 3) Choose your number of plays. To be interesting, please make it strictly
#    greater than 25:n > 25.
# --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --
#
#### n.plays <- 100 ----
n.plays <- 100


### Requirement 4:  Write a simulation... ----
# 4) Write a simulation in R that "plays roulette" using the roulette
#    probabilities and payouts determined by your bet. The simulation should
#    give your total net gain after n plays, possibly negative, assuming you
#    make the same bet each play.
# --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --
#
roulette.simulation <- function(risk = 3, plays = 30, reps=100){
  simulation.roulette <-  1:reps
  for(k in simulation.roulette){
    simulation.roulette[k]<-sum(sample(x=c(2*risk, -risk), # Column risk
                                prob=c(12/38, 26/38), # Column bet
                                size=plays, replace=TRUE))
  }
  simulation.roulette
}
roulette.simulation()


### Requirement 5:  Repeat the simulation... ----
# 5) Repeat the simulation at least 10,000 times.
# --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --
#
roulette.simulation(risk=3,plays=40,reps=10000)


### Requirement 6:  Make a histogram of the results... ----
# 6) Make a histogram of the results of the simulations. Use sensible bin
```

```
#    choices so that the rectangles are centered over the integers and so the
#    histogram is easy to see and read. It is okay if there are “gaps” over
#    impossible values.
# --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --
#
roulette.simulation <- function(risk = 3, plays = 30, reps=100, bins=50){
  simulation.roulette <-  1:reps
  for(k in simulation.roulette){
    simulation.roulette[k]<-sum(sample(x=c(2*risk, -risk), # Column risk
                                prob=c(12/38, 26/38), # Column bet
                                size=plays, replace=TRUE))
  }
  hist(simulation.roulette, col="white", prob=TRUE,
      breaks=seq(
        from=min(simulation.roulette),
        to=max(simulation.roulette),
        length.out=bins))
}
roulette.simulation(risk=3,plays=40,reps=10000,bins=80)


### Requirement 7:  Describe the bet you made... ----
# 7) In the document, or in the R plot window, briefly describe the bet you made
#    each play.
# --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --
#
roulette.simulation <- function(risk = 3, plays = 30, reps=100, bins=50){
  simulation.roulette <-  1:reps
  for(k in simulation.roulette){
    simulation.roulette[k]<-sum(sample(x=c(2*risk, -risk), # Column risk
                                prob=c(12/38, 26/38), # Column bet
                                size=plays, replace=TRUE))
  }
  hist(simulation.roulette, col="white", prob=TRUE, #plot = FALSE,
      breaks=seq(
        from=min(simulation.roulette),
        to=max(simulation.roulette),
        length.out=bins),
      main="Roulette",
      xlab= paste("Net gain ($) after",plays,"plays, risking $",risk,"on a column"))
}

# Final Histogram
roulette.simulation(risk=my.money.each.play,plays=n.plays,reps=10000,bins=80)

# Save the results of Simulation 2
png(filename = "./figures/roulette.simulation.png", width = 500,height = 500)
  roulette.simulation(risk=my.money.each.play,plays=n.plays,reps=10000,bins=80)
dev.off()

# --- --- --- --- ---
# rm(my.money.each.play,n.plays,roulette.simulation)
# --- --- --- --- ---


# _ ----

## Simulation 3: RATIO ----
# For the data set nb10, convert to micrograms under 10 grams:
```

```
#                    nb10data <- (10^6)*(10 - nb10$weight)
# Use nb10data as the "population."

### Data: nb10.rds ----
nb10 <- readRDS("nb10.rds")
nb10data <- (10^6)*(10 - nb10$weight)


### Requirement 1:  Describe the bet you made... ----
# 1) Choose a small sample size; for project purposes, say, 5 ≤ n ≤ 13.
# --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --
#
#### Sample size:  10


### Requirement 2:  Describe the bet you made... ----
# 2) Draw at random, with replacement (to simulate "independent measurements")
#    n times from nb10data.
# --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --
#
independent.measurement <- sample(nb10data,size=7,replace=TRUE)
independent.measurement
#


### Requirement 3:  Apply a t-ratio... ----
# 3) For your small random sample of n numbers, apply the formula, called a
#   t-ratio,
#            t =X̄ − 404.59 / [(√n × s)/n].
#
#    Here, X̄ is the average of the n numbers in your sample. Then, s is the
#    so-called "sample SD" of the n numbers in your sample, computed using the
#    built-in R function sd. Note that the ratio formula uses the same idea of
#    "standardizing" data values as in Lesson 02, except using expected value
#    and standard error for a random variable, instead of average and SD for a
#    data value.
# --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --
#
(avg(independent.measurement) - 404.59) /(( sqrt(7) * sd(independent.measurement) ) / 7 )


### Requirement 4:  Write a simulation... ----
# 4) Write a simulation in R that repeats the chance procedure and applies the
#    t-ratio formula to the random sample at least 10,000 times.
# --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --
#
t.ratio.simulation<-function(n=7,reps=10000){
  simulated.data<-1:reps
  for (k in simulated.data) {
    independent.measurement <- sample(nb10data,size=n,replace=TRUE)
    simulated.data[k]<-(avg(independent.measurement) - 404.59)/
      (( sqrt(n) * sd(independent.measurement) ) / n )
  }
  simulated.data
}
# Test
t.ratio.simulation()
```

```
### Requirement 5:  Make a histogram of the simulation... ----
# 5) Make a histogram of the simulation results to get an idea of what the
#    probability histogram of the random variable t looks like. Use 200 or more
#    bins.
# --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --
#
t.ratio.simulation<-function(n=7,reps=10000,bins=220){
  simulated.data<-1:reps
  for (k in simulated.data) {
    independent.measurement <- sample(nb10data,size=n,replace=TRUE)
    simulated.data[k]<-(avg(independent.measurement) - 404.59)/
      (( sqrt(n) * sd(independent.measurement) ) / n )
  }
  hist(simulated.data, col="white", prob=TRUE,
       breaks=seq(
         from=min(simulated.data),
         to=max(simulated.data),
         length.out=bins),
       yaxt='n',
       ylab=NULL)
}
# Test
t.ratio.simulation()


### Requirement 6:  Add the standard normal curve... ----
# 6) Add a graph of the standard normal curve to the histogram with
#                    curve(dnorm(x), add=TRUE, lty=2)
# --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --
#
t.ratio.simulation<-function(n=7,reps=10000,bins=220){
  simulated.data<-1:reps
  for (k in simulated.data) {
    independent.measurement <- sample(nb10data,size=n,replace=TRUE)
    simulated.data[k]<-(avg(independent.measurement) - 404.59)/
      (( sqrt(n) * sd(independent.measurement) ) / n )
  }
  hist(simulated.data, col="white", prob=TRUE,
       breaks=seq(
         from=min(simulated.data),
         to=max(simulated.data),
         length.out=bins),
       yaxt='n',
       ylab=NULL)
  curve(dnorm(x), add=TRUE, lty=2)
}
# Test
t.ratio.simulation()

### Requirement 7:  Add another theoretical curve... ----
# 7) The histogram is not quite standard normal, especially in the “tails.” The
#     probability histogram for the random variable t has “thicker tails” than
#     a standard normal histogram. Add another theoretical curve to the
#     histogram:
#               curve(dt(x, df=n-1), add=TRUE)
#
#     Replace n with your chosen sample size.
# --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --- --
```

```
#
t.ratio.simulation<-function(n=7,reps=10000,bins=220){
  simulated.data<-1:reps
  for (k in simulated.data) {
    independent.measurement <- sample(nb10data,size=n,replace=TRUE)
    simulated.data[k]<-(avg(independent.measurement) - 404.59)/
      (( sqrt(n) * sd(independent.measurement) ) / n )
  }
  hist(simulated.data, col="white", prob=TRUE,
       breaks=seq(
         from=min(simulated.data),
         to=max(simulated.data),
         length.out=bins),
       yaxt='n',
       ylab=NULL,
       main=paste("t-ratio on nb10 small random samples; I used n =",n,
                  "\nThe dashed curve is standard normal z, and the solid curve is t"),
       cex.main=1,
       xlab= paste("value of t"))
  curve(dnorm(x), add=TRUE, lty=2)
  curve(dt(x, df=n-1), add=TRUE)
}
# Final Histogram
t.ratio.simulation(n=10,reps=10000,bins=220)

# Save result of Simulation 3
png(filename = "./figures/t.ratio.simulation.png", width = 500,height = 500)
  t.ratio.simulation(n=10,reps=10000,bins=220)
dev.off()
```